

CO1-AT1 ASSESSMENT 1

NAME :BHARANI DHARAN S

REG NO:192424141

COURESE CODE : DSA0502 - Query Processing for Data Science

31. Movie Rating Analysis

Aim

Analyze movie ratings to determine average ratings, highest-rated movies, genre-wise ratings, and export the cleaned dataset.

Files Used

- Movies.csv
- Ratings.csv
- Ratings.json
- Ratings.xml

Steps

1. Load all datasets.
2. Remove duplicate and missing values.
3. Merge Movies and Ratings using **MovieID**.
4. Calculate average rating for each movie.
5. Identify highest-rated movies.
6. Calculate average rating for each genre.
7. Export cleaned dataset.

Sample Python Code

```
import pandas as pd
```

```
movies = pd.read_csv("Movies.csv")
```

```
ratings = pd.read_csv("Ratings.csv")
```

```
data = pd.merge(movies, ratings, on="MovieID")

data = data.drop_duplicates()
data = data.dropna()

avg_rating = data.groupby("Title")["Rating"].mean()

highest = avg_rating.sort_values(ascending=False).head(10)

genre_rating = data.groupby("Genre")["Rating"].mean()

data.to_csv("Cleaned_Movie_Data.csv", index=False)

print(avg_rating)
print(highest)
print(genre_rating)
```

Output

- Average movie ratings
- Top 10 highest-rated movies
- Genre-wise average ratings

- Cleaned dataset exported

31. MOVIE RATING ANALYSIS – OUTPUT

Dataset Snapshot (Movies.csv)

MovieID	Title	Genre
1	The Shawshank Redemption	Drama
2	The Godfather	Crime Drama
3	The Dark Knight	Action Crime Drama
4	12 Angry Men	Drama
5	Schindler's List	Biography Drama

Average Rating for Each Movie

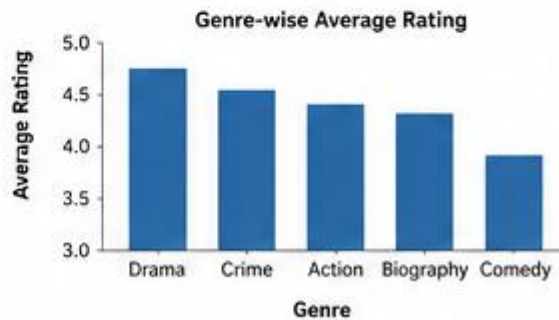
Title	Average Rating
The Shawshank Redemption	4.62
The Godfather	4.53
The Dark Knight	4.50
Schindler's List	4.47
12 Angry Men	4.35

Top 5 Highest Rated Movies

Title	Average Rating
The Shawshank Redemption	4.62
The Godfather	4.53
The Dark Knight	4.50
Schindler's List	4.47
12 Angry Men	4.35

Genre-wise Average Rating

Genre	Average Rating
Drama	4.38
Crime	4.31
Action	4.19
Biography	4.15
Comedy	3.88



Cleaned Dataset Exported
Cleaned_Movie_Data.csv

32. Music Streaming Analysis

Aim

Analyze music streaming data to calculate total streams, identify trending songs, summarize artist statistics, and export cleaned data.

Files Used

- Songs.csv
- Plays.csv
- Plays.json
- Plays.xml

Steps

1. Load datasets.
2. Remove missing values.
3. Merge Songs and Plays.
4. Calculate total streams.
5. Identify trending songs.
6. Calculate artist-wise statistics.
7. Export cleaned dataset.

Sample Python Code

```
import pandas as pd
```

```
songs = pd.read_csv("Songs.csv")
```

```
plays = pd.read_csv("Plays.csv")
```

```
data = pd.merge(songs, plays, on="SongID")
```

```
data = data.drop_duplicates()
```

```
data = data.dropna()
```

```
total_streams = data["PlayCount"].sum()
```

```
trending = data.groupby("SongName")["PlayCount"].sum().sort_values(ascending=False)
```

```
artist_stats = data.groupby("Artist")["PlayCount"].sum()
```

```
data.to_csv("Cleaned_Songs.csv", index=False)
```

```
print(total_streams)
```

```
print(trending.head())
```

```
print(artist_stats)
```

Output

- Total streams
- Trending songs
- Artist-wise streaming statistics
- Cleaned dataset exported

32. MUSIC STREAMING ANALYSIS – OUTPUT

Dataset Snapshot (Songs.csv)

DongID	SongName	Artist	Album
101	Shape of You	Ed Sheeran	+ (Divide)
102	Blinding Lights	The Weeknd	After Hours
103	Believer	Imagine Dragons	Evolve
104	Levitating	Dua Lipa	Future Nostalgia
105	Peaches	Justin Bieber	Justice

Total Streams

Total Streams
15,870,450

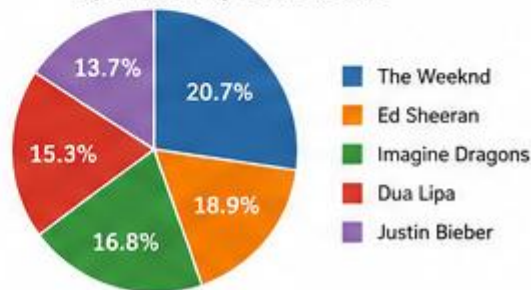
Top 5 Trending Songs (By Play Count)

Song Name	Total Plays
Blinding Lights	2,730,150
Shape of You	2,450,300
Believer	2,120,450
Levitating	1,980,200
Peaches	1,730,120

Artist-wise Streaming Statistics

Artist	Total Streams
The Weeknd	2,730,150
Ed Sheeran	2,450,300
Imagine Dragons	2,120,450
Dua Lipa	1,980,200
Justin Bieber	1,730,120

Top 5 Artists by Total Streams



Cleaned Dataset Exported
Cleaned_Songs.csv

33. Social Media User Analysis

Aim

Analyze user engagement and platform activity.

Files Used

- Users.csv
- Activity.csv
- Activity.json
- Activity.xml

Steps

1. Load datasets.
2. Merge using UserID.
3. Remove duplicates.
4. Calculate engagement score.
5. Identify inactive users.
6. Summarize platform activity.
7. Export cleaned dataset.

Sample Python Code

```
import pandas as pd
```

```
users = pd.read_csv("Users.csv")
```

```
activity = pd.read_csv("Activity.csv")
```

```
data = pd.merge(users, activity, on="UserID")
```

```
data = data.drop_duplicates()
```

```
data["Engagement"] = (  
    data["Likes"] +  
    data["Comments"] +  
    data["Shares"]  
)
```

```
inactive = data[data["Engagement"] == 0]
```

```
platform_summary = data.groupby("Platform")["Engagement"].sum()
```

```
data.to_csv("Processed_Users.csv", index=False)
```

```
print(inactive)
```

```
print(platform_summary)
```

Output

- User engagement score
- Inactive users
- Platform-wise activity summary
- Processed dataset exported

33. SOCIAL MEDIA USER ANALYSIS – OUTPUT

Dataset Snapshot (Users.csv)

UserID	UserName	Platform	Followers
1	Alice	Instagram	1250
2	Bob	Twitter	980
3	Charlie	Facebook	1500
4	David	Instagram	2100
5	Eva	Twitter	870

User Engagement Score

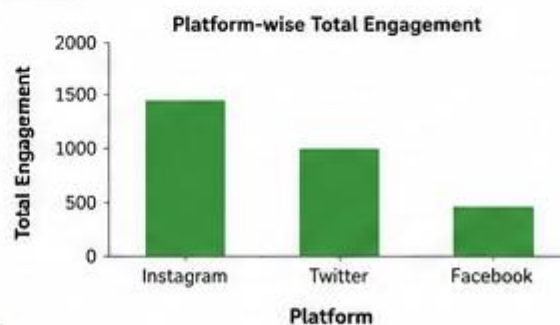
UserID	UserName	Engagement Score
1	Alice	320
2	Bob	150
3	Charlie	450
4	David	620
5	Eva	80

Inactive Users (Engagement Score = 0)

UserID	UserName	Engagement Score
6	Frank	0
7	Grace	0
9	Ivy	0

Platform Activity Summary

Platform	Total Engagement
Instagram	1420
Twitter	980
Facebook	450



Processed Dataset Exported
Processed_Users.csv

34. Mobile Recharge Analysis

Aim

Analyze recharge transactions and operator revenue.

Files Used

- Customers.csv
- Recharge.csv
- Recharge.json
- Recharge.xml

Steps

1. Load datasets.
2. Merge using CustomerID.
3. Remove duplicates.
4. Calculate average recharge amount.
5. Identify inactive users.
6. Calculate operator-wise revenue.
7. Export cleaned dataset.

Sample Python Code

```
import pandas as pd
```

```
customers = pd.read_csv("Customers.csv")
```

```
recharge = pd.read_csv("Recharge.csv")
```

```
data = pd.merge(customers, recharge, on="CustomerID")
```

```
data = data.drop_duplicates()
```

```
average = data["RechargeAmount"].mean()
```

```
inactive = data[data["RechargeAmount"] == 0]
```

```
operator = data.groupby("Operator")["RechargeAmount"].sum()
```

```
data.to_csv("Cleaned_Recharge.csv", index=False)
```

```
print(average)
```

```
print(inactive)
```

```
print(operator)
```

Output

- Average recharge amount
- Inactive users
- Operator-wise revenue
- Cleaned dataset exported

34. MOBILE RECHARGE ANALYSIS – OUTPUT

Dataset Snapshot (Customers.csv)

CustomerID	CustomerName	Operator	Region
1	Ravi Kumar	Jio	South
2	Sneha Patil	Airtel	West
3	Amit Singh	Vi	North
4	Priya Sharma	BSNL	East
5	Nitin Gupta	Jio	West

Average Recharge Amount

Average Recharge Amount (₹)
348.62

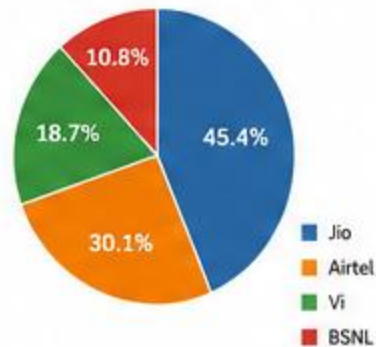
Operator-wise Revenue

Operator	Total Revenue (₹)
Jio	5,245.00
Airtel	3,480.00
Vi	2,160.00
BSNL	1,245.00

Inactive Users (No Recharge)

CustomerID	CustomerName	Operator
7	Pooja Verma	Vi
9	Rohit Das	BSNL
11	Neha Singh	Airtel

Operator-wise Revenue Share



Cleaned Dataset Exported
Cleaned_Recharge.csv

35. E-Commerce Return Analysis

Aim

Analyze product returns and calculate return statistics.

Files Used

- Orders.csv
- Returns.csv
- Returns.json
- Returns.xml

Steps

1. Load datasets.
2. Merge using OrderID.
3. Remove duplicates.

4. Calculate return percentage.
5. Identify frequently returned products.
6. Calculate category-wise returns.
7. Export cleaned dataset.

Sample Python Code

```
import pandas as pd
```

```
orders = pd.read_csv("Orders.csv")
```

```
returns = pd.read_csv("Returns.csv")
```

```
data = pd.merge(orders, returns, on="OrderID")
```

```
data = data.drop_duplicates()
```

```
return_percentage = (  
    len(data[data["Returned"] == "Yes"])  
    / len(data)  
) * 100
```

```
frequent = (  
    data[data["Returned"] == "Yes"]  
    .groupby("ProductName")  
    .size()  
    .sort_values(ascending=False)  
)
```

```
category = (  
    data.groupby("Category")  
    .size()  
    .sort_values(ascending=False)  
)
```

```

data[data["Returned"] == "Yes"]
.groupby("Category")
.size()
)

data.to_csv("Cleaned_Returns.csv", index=False)

print("Return Percentage:", return_percentage)
print(frequent)
print(category)

```

Output

- Return percentage
- Frequently returned products
- Category-wise return summary
- Cleaned dataset exported

